

[← Back to Articles](#)

[🔗](#) The PR you would have opened yourself

Published April 16, 2026

[Update on GitHub](#)

▲ Upvote **68**



Pedro Cuenca

[pcueng](#)

Follow



Awni Hannun

[awni](#)

Follow

MLX

[mlx-community](#)

Making transformers models available in mlx-lm using a Skill and test harness

[🔗](#) TL;DR

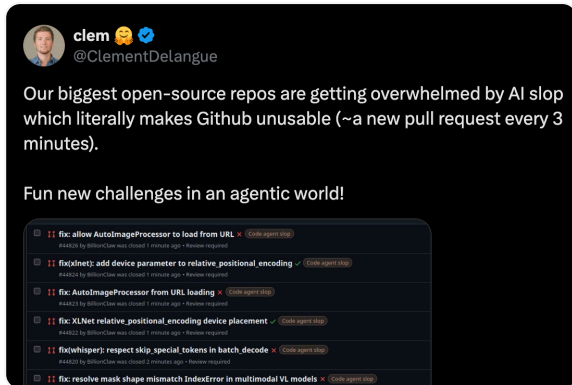
We provide a **Skill** and a **test harness** to help port language models from transformers to mlx-lm, so they become (almost) instantly available the moment they are added to transformers. The Skill is designed to support contributors and reviewers as an aide, not an automation. We explain why we did it, how, and comment about how to meaningfully contribute to open source in the age of agents.

[🔗](#) The advent of code agents

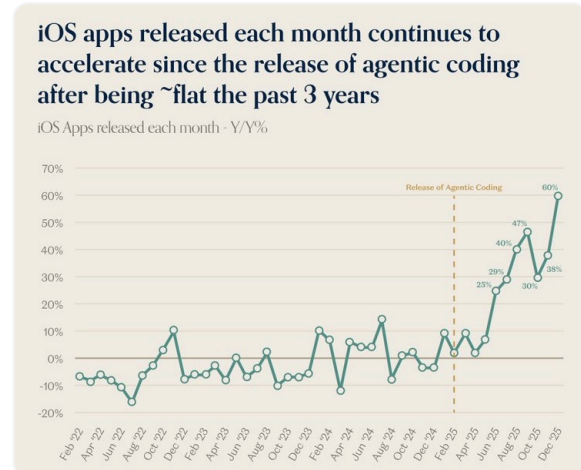
In 2026, code agents started to actually work. What used to be auto-completion at the side of your editor turned into a system that one-shots reasonable solutions from brief specifications. The generated code usually works out of the box, covers what you asked for, and makes reasonable assumptions about details you didn't specify. This is great. As Jensen Huang puts it, we've instantly gone from 30 million to one billion coders in the world. Creative minds are unleashed.

But it forces us to rethink open source.

Take the transformers library as an example. It has hundreds of contributors, is used in thousands of projects, has been downloaded over a billion times. Suddenly, anyone with an agent can instruct it to find some open issue, fix it, and submit a PR. And that's exactly what's happening. Those people feel happy because they are contributing to a great library, but the sad reality is that, most of the time, they don't realize they are not.



Source: [@ClementDelangue](#)



Source: a16z / Sensor Tower

Why not? There are two assumptions that agent-generated PRs usually miss.

- **Codebases like transformers care deeply about the code.** It's cool to build projects where it doesn't matter what the code looks like, but transformers is not one of them. Being used by thousands of people, transformers is primarily built as a human-to-human communication method, through code. Model files read top to bottom, because we want practitioners to understand them without jumping through complex abstractions. This permeates throughout the library design and is the reason why, for example, we favor flat hierarchies.
- **Agents don't have that context.** Because design decisions are not explicit, agents suggest refactors to "improve" the codebase by following "best practices", without realizing they are breaking implicit contracts between the library and its users. They are verbose, generalize too early, don't notice when a change affects other areas, introduce subtle bugs, break performance. They are also sycophantic, and accept any idea as good and follow it through diligently,

including ones a maintainer would have pushed back early on with a terse comment.

A small number of maintainers still has to read every PR, understand it, decide if the design direction is right, identify side effects, and write feedback. PR volume has gone up tenfold, but the amount of maintainers has not (and cannot, because team coordination does not scale).

🔗 What does this have to do with MLX?

Transformers is one of the first projects to feel this pressure because of sheer volume, but the same dynamic is happening everywhere. As an example from a different domain, App Store reviewers are swamped because anyone can now build and submit an app, so many do.

The same logic applies to MLX: their maintainers care deeply about the code and read every PR carefully. We wanted to see whether agents could *help contributors* land high-quality model ports fast, and at the same time *support reviewers* in their work. Not only do we aspire to produce PRs that could have come from a careful human submission, but we also provide additional artifacts to increase the signal: generation examples, numerical comparisons, and a separate non-agentic test harness for reproducibility.



Another connection between transformers and MLX is that, most times, mlx-lm models are ported from transformers implementations. Because transformers focuses on clarity and readability, it has become the source of truth for model definitions. Downstream contributors wait until the transformers implementations

are ready before they port to other frameworks. As a side effect, this is an excellent environment for an agent because it naturally limits the scope: rather than creating an implementation from scratch, the agent relies on transformers code as the source of truth.

This approach supports our goal: when a model lands in transformers, it should be available on MLX shortly after.

🔗 What we did

We built a Skill that mlx-lm contributors can use to port a model from transformers to MLX. Given a prompt like "convert the olmo_hybrid architecture to MLX", the Skill sets up a virtual environment to work on, discovers and downloads the relevant models from the Hub, reads the transformers modeling code, writes the MLX implementation, and runs a battery of tests. If results don't look right, it debugs and iterates, and does not declare success until it's satisfied.

We designed it to be useful to reviewers as much as contributors.

For the contributor, the Skill of course handles all the scaffolding: finding model variants on the Hub, diffing their configs to spot parameters that vary across model variants, downloading checkpoints, setting up editable installs of both mlx-lm and transformers. But it also handles the more difficult modeling tasks. It pays attention to salient architecture details and verifies sensitive areas, like RoPE configurations, that may result in hard-to-find bugs. It detects when the config doesn't declare a dtype and infers it from the safetensors metadata header. It runs per-layer comparisons between transformers and MLX to pinpoint exactly where divergence occurs. These are the kinds of checks that only someone with porting experience would think to run.

For the reviewer, the Skill produces a PR that is upfront about being agent-assisted, but does look like a careful human submission. Reviewers will see that the code follows mlx-lm conventions: idiomatic solutions, no unnecessary comments, no speculative abstractions, no modifications to shared utilities without explicit approval. Given that the code is agent-assisted, we try to include *more* data than the

median PR, to provide as much signal as possible. The PR body includes a report with a summary of the variants and their architectural differences, generation examples, numerical comparisons, dtype verification, per-layer comparisons against the transformers baseline. The PR always discloses that it was agent-assisted, and the Skill will not open it until the contributor has accepted the results.

For verification, the Skill generates a test manifest for a separate, non-agentic test harness that is, by design, easily reproducible and not subject to LLM hallucinations or complacency ([more on this below](#)).

🔗 How we did it

Skills are recipes for agents: simple text files with guidelines that steer the model through a complex task. They are not magic; you can achieve the same results via prompting and iteration. But they provide *consistency* (every run follows the same process, whereas different people would prompt differently), minimize ambiguity and serve as documentation: anyone can read the Skill to understand what it does, identify missing cases and suggest improvements.

We bootstrapped the Skill by porting a model ourselves, in conversation with Claude. I asked it to port GLM 4.7 from transformers to mlx-lm, giving instructions as I would during a normal session. One trick: I pointed Claude at a checkout of mlx-lm from which I had deleted the already-existing implementation, so I could compare the output against the ground truth. After a few iterations I had a working implementation, a conversation that revealed how Claude approached the problem, and the first draft of the Skill, which Claude created as a summary of the process. I edited it heavily, and incorporated the learnings from [@gabegoodhart](#), who kindly shared [their own porting conversation](#) for a different model 🙌.

We repeated this loop several times and the Skill grew. On the technical side, we covered stuff such as [RoPE bugs](#) that may produce plausible output that degrades with long sequences, float32 precision contamination that silently kills inference speed (you'd be surprised how frequently these things happen!), config fields that vary across model variants in ways the implementation must handle, distributed inference for super large models that don't fit on a single machine. We taught it how

to invoke the `hf` CLI to discover and download models. Most importantly, we instructed it to run the tests that experienced porters would, and to not declare success until they pass.



Source: [@Prince_Canuma](#)

On the cultural side, we covered *softer* characteristics and explained the conventions that make a PR easy to review: don't use comments to explain code (the reviewer has to parse the comment *and* the code 🧑), never propose refactors, don't touch shared utilities without asking. These rules cost the agent nothing but save the reviewer lots of time.

The end result: the contributor types a prompt, and the Skill produces a PR like [this one](#), plus a test manifest for the external test harness.

🔗 Test harness

The Skill shares a comprehensive results report as part of the PR. All these come from tests the agent runs during conversion, but we didn't want the reviewer to take a leap of faith to accept them. To go a step forward, we created a separate, non-agentic test harness that runs systematic tests on the converted code. This brings a couple of benefits:

- Removes uncertainty about the LLM hallucinating results, or being too complacent about them.

- Guarantees reproducibility: anyone can download the test harness repo and run the tests.
- Documentation and transparency. All results are saved at various levels: summary reports, per-model details, raw inputs/outputs saved as JSON files. The tests are also copied to results folders so we know what we ran even if we make changes to the harness in the future.

The test harness is not a CI gate. Some checks are straightforward (is the output dtype correct?), but most are qualitative. Is it normal that a pre-trained model repeats itself in long sequences? Is a 4% relative logits difference against the transformers baseline acceptable? These are judgement calls based on experience with similar architectures. The harness provides useful signal, but it's the reviewer and contributor who still have to make the call.

🔗 How to use the Skill

The Skill is designed for the people who are already opening mlx-lm model PRs, or who would do it manually on their own. It's not meant for mass consumption, because PRs to mlx-lm are rarely accepted on sight. The typical cycle is: contributor opens a PR, reviewers point out improvements, both sides iterate until the quality bar is met. If this is true for expert submissions, it will remain true for agent-assisted ones.

If you're not prepared to engage in that cycle, you probably shouldn't be opening a PR. The reviewers will make an effort to understand your code (even knowing it was agent-assisted), so you should do the same. Own the code, and be ready to incorporate their feedback. In particular, don't hand reviewer comments back to an agent and post whatever it produces. LLMs double down on their decisions, go on tangents, and don't push back effectively. Once you engage with the reviewer, this becomes a person-to-person conversation, so it's your turn to discuss and be respectful of the time they put in.

You can also use the Skill to learn; you don't need to submit anything until your confidence and experience build. Read the Skill to identify problem areas you

weren't aware of: it contains nearly 15 thousand words among the skill file, reference docs and utility scripts. Point it to your own fork of `mlx-lm`, try a conversion, and compare your output against the accepted implementation once it lands in the official repo. If you do this a few times, you'll learn a lot about transformers, MLX, and language model architectures.

If you're ready:

```
uv run https://raw.githubusercontent.com/huggingface/transformers-to-mlx  
uvx hf skills add --claude
```

We developed and tested the Skill using Claude Code. The same approach would work with Codex or other coding agents, but we haven't tested them. If you try the Skill in a different environment, please let us know how it goes!

🔗 Next steps and known shortcomings

The Skill works well for LLMs in `mlx-lm`, but there's plenty of room to grow.

🔗 What's next

- **mlx-vlm.** Vision-language models live in a separate repo with different conventions. Beyond the modeling code, `mlx-vlm` requires *processors* to handle image pre-processing before the LLM sees the input. We're looking forward to collaborating with Prince Canuma to help him do what he does.
- **llama.cpp.** Some of the same challenges apply. Processors require image processing algorithms to be replicated in C++, and numerical differences are unavoidable. This is an area where a tightly scoped agent might help.
- **The test harness.** We want to expand the test battery and potentially explore safe automation to run tests automatically on our infra.

🔗 What doesn't work yet

- **Shared utilities in mlx-lm.** mlx-lm is less strict than transformers about extracting common patterns into shared functions. The Skill is purposefully biased towards self-contained model files (same as transformers), but reviewers regularly ask for refactors to move repeated code into shared modules.
- **VLMs and other architectures**, as noted above.
- **Quantized model uploads.** The Skill tests quantization but doesn't upload quantized models to the Hub. We think it doesn't make sense to upload while the PR is being reviewed, but we could create a flow to do it later.
- **Thinking tests.** No thinking-specific tests have been designed yet. The Skill will convert and verify generations from these models, but won't validate the thinking structure.

🔗 Conclusion

The bottleneck in open source is not typing speed: it's understanding the codebase to change it without breaking the implicit and explicit contracts with users. Agents can help in this process, if we teach them what matters. We explored what this looks like in the context of mlx-lm, and hope it's useful for contributors and reviewers to land high-quality model conversions faster!

🔗 Resources

Contributions:

- [transformers-to-mlx Skill repo](#)
- [Test Harness repo](#)
- [Example agent-assisted conversion against a fork](#)

The libraries:

- [mlx-lm, the target library](#)

- [transformers, the source of truth for modeling code](#)

Background:

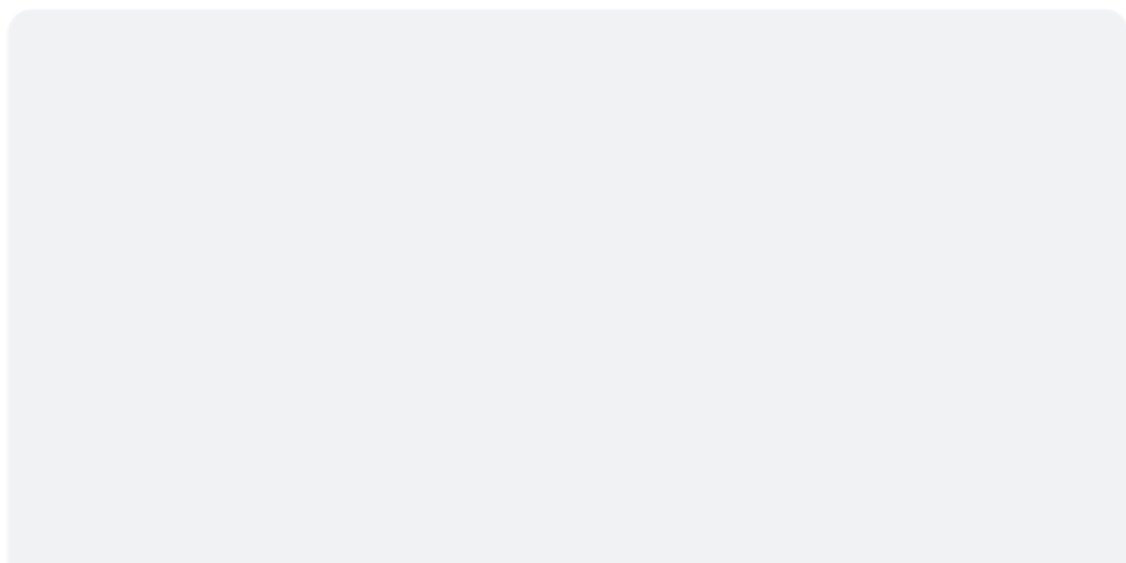
- [Claude Code Skills docs](#)
- [Transformers design philosophy](#)
- [The Transformers Library: standardizing model definitions](#)

🔗 Thank you!

Thanks a lot to [Ben](#), [Shaun](#), [Aritra](#) for reading previous versions of this post and making it so much better 🙌🙌

We are incredibly indebted to Apple for making MLX an open-source project, and to the community for instantly recognizing its value and contributing enthusiastically 🙌

More Articles from our Blog



Llm

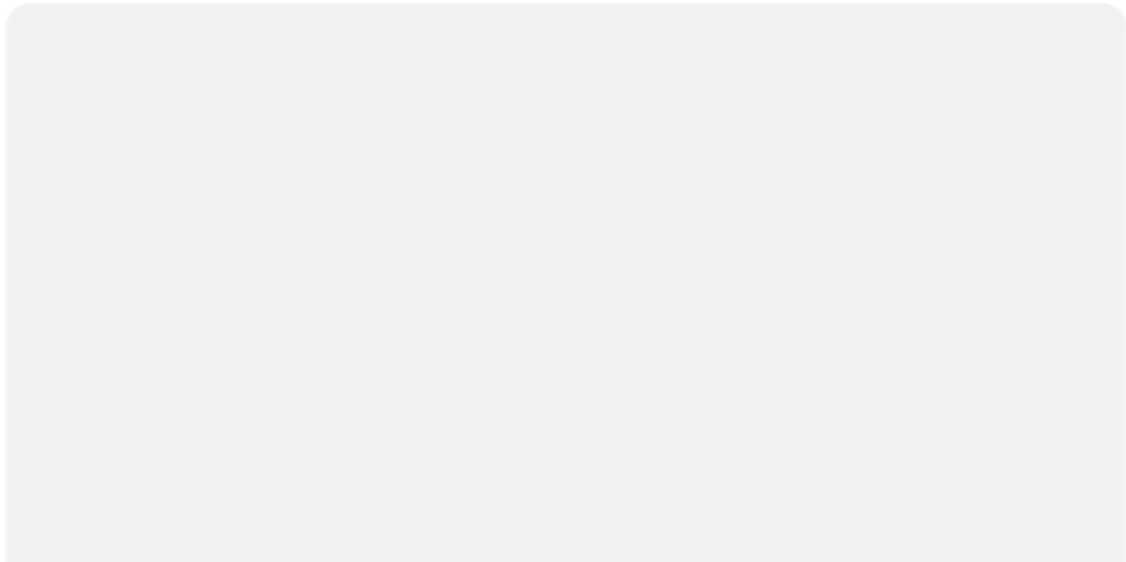
Moe

Long-Context

DeepSeek-V4: a million-token context that agents can actually use

 burtenshaw

△ 41 April 24, 2026



Cybersecurity Open-Source Community

AI and the Future of Cybersecurity: Why Openness Matters

 meg, et. al.

△ 36 April 21, 2026

Community

Edit

Preview

Start discussing this article

 Tap or paste here to upload images

 Comment

[Sign up](#) or [log in](#) to comment